

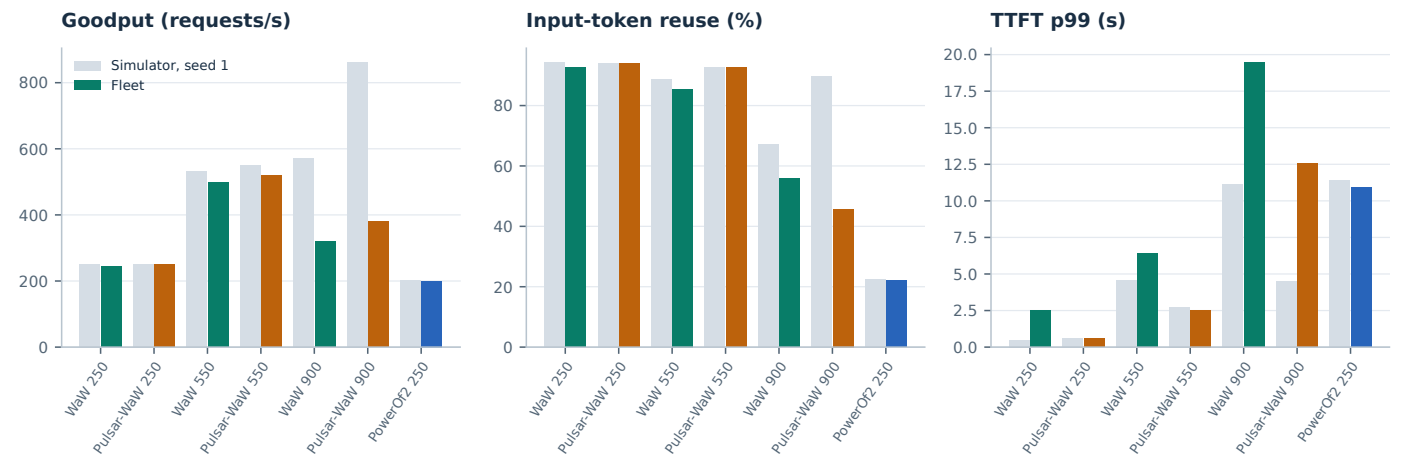
SUMMARY / DECISION

Pulsar-WaW leads; the fleet breaks earlier than the simulator

We compared WaW and Pulsar-WaW, with PowerOf2 and round-robin as baselines, on a heterogeneous fleet of 20 backends with 2-20 GPUs each (220 GPUs). The discrete-event simulator and the five-region dev clusters ran the same workload, engine model and routing configuration.

Pulsar-WaW matched or beat WaW at every rate on both the simulator and the fleet. At 550 requests/s on the fleet it met the SLO for 100.0% of requests with 92.6% reuse; WaW reached 99.6% with 85.5% reuse and 148 timeouts.

Policy	Nominal RPS	Offered RPS fleet / sim	Goodput fleet / sim	SLO met	Reuse	TTFT p99	Fleet failures
WaW	250	245 / 249	244.8 / 249.1	100.0% / 100.0%	92.5% / 94.3%	2.52 s / 0.49 s	0
Pulsar-WaW	250	250 / 249	249.6 / 249.3	100.0% / 100.0%	94.0% / 94.0%	0.60 s / 0.58 s	0
WaW	550	501 / 533	498.9 / 532.0	99.6% / 99.7%	85.5% / 88.6%	6.39 s / 4.59 s	148
Pulsar-WaW	550	520 / 550	520.2 / 549.8	100.0% / 100.0%	92.6% / 92.7%	2.53 s / 2.74 s	0
WaW	900	532 / 654	319.5 / 571.7	60.1% / 87.4%	55.8% / 67.2%	19.47 s / 11.14 s	26,151
Pulsar-WaW	900	597 / 862	380.2 / 861.3	63.7% / 100.0%	45.8% / 89.5%	12.56 s / 4.46 s	48,815
PowerOf2	250	204 / 204	200.2 / 200.7	98.3% / 98.2%	22.1% / 22.3%	10.91 s / 11.37 s	185



Fleet runs (colored) against simulator seed 1 (grey) for the same policy and nominal rate. Goodput counts successful requests whose TTFT met the 10 s SLO, per second of the 240 s measured window.

Up to 550 requests/s, fleet SLO attainment, reuse and tail latency match the simulator closely for Pulsar-WaW, and for PowerOf2 at 250; fleet goodput is up to 5% lower because fewer turns were offered. WaW does worse on the fleet than simulated because its smallest backends overload sooner. At 900 requests/s both affinity policies collapse on the fleet; the simulator shows WaW degrading (87.4-89.4% SLO) and Pulsar-WaW holding 100.0%. Each fleet point is one run.

STUDY DESIGN / WHAT WAS HELD EQUAL

One workload, one engine model, two environments

Region	Stargates	Backend GPU workers
usw2	3	7, 4, 2, 3
ue1	3	14, 6, 12, 9
ew1	3	19, 8, 10, 13
an1	3	11, 16, 15, 20
as2	3	18, 17, 11, 5

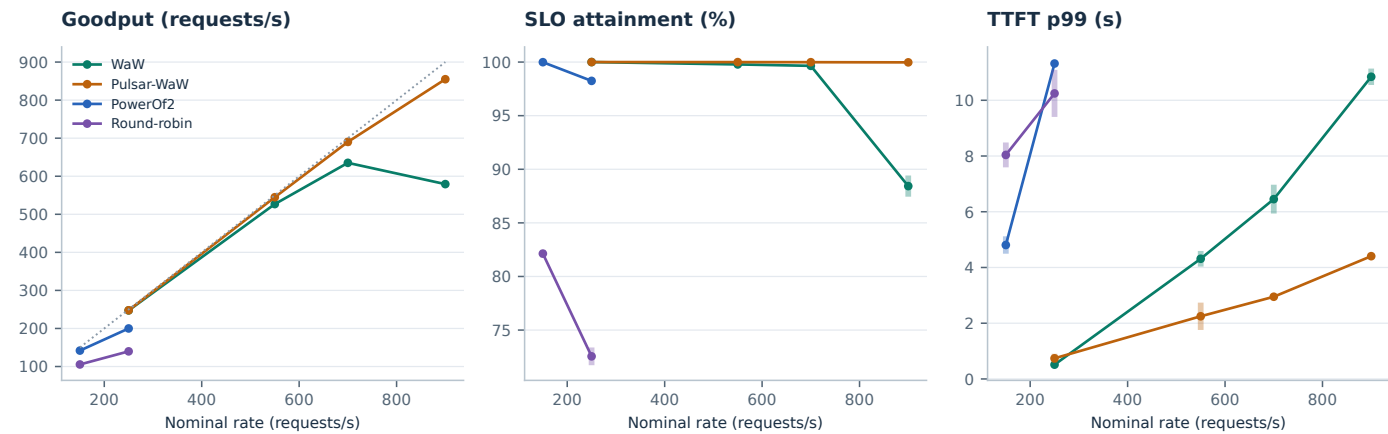
Control	Setting
Engine model	Batched mock engine shared by MockDynamo and the simulator: 25 sequences and 2,048 batched tokens per GPU worker; step = 4.0 ms + 0.085 ms per decoding sequence + 0.05 ms per prefill token; 1,000,000 KV tokens per GPU; perfect KV routing between workers
Pylon	Max engine concurrency = 25 x GPU workers; fallback-window input TPS (maximum, initial 20,000); queue-mismatch admission 10,000 ms / 4.0; change-driven stats coalesced to 10 ms; engine stats stream disabled
Client	10 s TTFT SLO and maximum wait; 30 s timeout; up to 3 attempts per turn, 1 s backoff
Workload	Growing sessions: 4,000-token system prompt, 200-2,000 user tokens and 128-256 output tokens per turn, 5-40 turns, 5 s mean think time; each turn's prompt includes all earlier turns and outputs
Timing	240 s warmup, then 240 s measured; equal traffic from all five regions
Reset	Fleet: all MockDC backends restarted before each run, then 20 of 20 registered on all 15 Stargate pods

Policy	Configuration
WaW	Cache-affinity ring (150 virtual nodes per backend), 1 affine backend, 300 ms wait, max_queued 4, utilization comparator
Pulsar-WaW	Maximum-TPS weighting; 300 ms affinity wait; bands widen every 300 ms (1, 2, 4 ... backends); max_queued 4; overflow only to free engine slots (fallback_max_queued 0); 4 s queue bounds
PowerOf2	Two random samples, TTFT comparator
Round-robin	No load or cache signal (simulator only)

Nominal rate sets the session arrival rate; later turns wait for the previous response plus think time. Slow responses therefore lower the offered rate, so every table reports offered rate next to goodput. Reuse is reused input tokens over input tokens of successful requests. Simulator results use seeds 1 and 2; fleet results use seed 1 and compare against simulator seed 1.

SIMULATOR / SEEDS 1 AND 2

Pulsar-WaW keeps reuse as load rises



Lines join the midpoints of the two seeds; vertical bars show the seed range. The dotted line in the goodput panel is goodput equal to nominal rate.

Policy	RPS	Offered	Goodput	SLO %	Reuse %	TTFT p50 s	TTFT p99 s	Failures
WaW	250	246-249	245.8-249.1	100.0	94.1-94.3	0.18	0.5	0
WaW	550	522-533	521.6-532.0	99.7-99.8	88.6-89.5	0.24-0.25	4.0-4.6	148-217
WaW	700	636-639	634.1-636.3	99.6-99.7	83.3-84.9	0.33-0.39	5.9-7.0	272-274
WaW	900	654-656	571.7-586.9	87.4-89.4	67.2-69.6	1.69-1.92	10.5-11.1	14,821-17,300
Pulsar-WaW	250	246-249	246.2-249.3	100.0	93.7-94.0	0.18-0.19	0.6-0.9	0
Pulsar-WaW	550	540-550	539.9-549.8	100.0	92.7-93.4	0.20	1.8-2.7	0
Pulsar-WaW	700	680-701	679.7-700.7	100.0	92.3-92.4	0.22	2.9-3.0	0
Pulsar-WaW	900	849-862	848.5-861.3	100.0	89.5-89.6	0.27-0.28	4.4-4.5	7-10
PowerOf2	150	138-146	137.5-146.2	100.0	31.6-33.0	0.88-0.97	4.5-5.1	0
PowerOf2	250	203-204	199.3-200.7	98.2-98.3	22.3-23.1	1.36-1.49	11.3-11.4	73-121
Round-robin	150	125-131	103.4-107.4	81.8-82.5	24.6-25.5	0.82-0.85	7.6-8.5	5,138-5,641
Round-robin	250	192-194	139.1-140.6	71.7-73.4	21.3	0.85	9.4-11.1	11,961-12,688

At 900 requests/s, Pulsar-WaW sustains 848.5-861.3 goodput with 89.5-89.6% reuse; WaW falls to 571.7-586.9 with 67.2-69.6% reuse. PowerOf2 and round-robin ignore cache placement and lose capacity early on this cache-heavy workload: PowerOf2 falls short at 250 requests/s and round-robin already at 150.

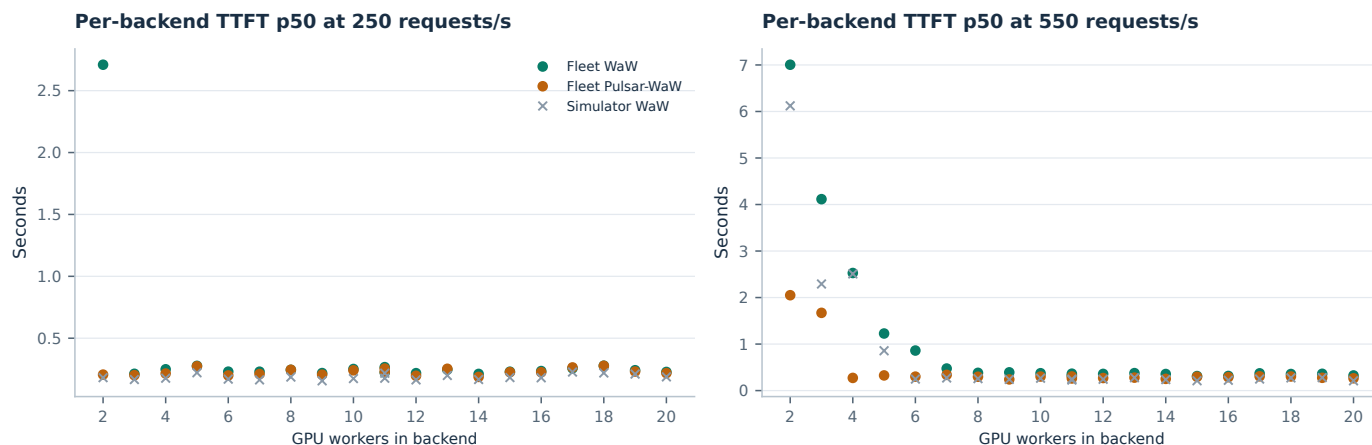
Ranges are the two observed seeds, not confidence intervals.

FLEET / MATCHED AGAINST THE SIMULATOR

The fleet agrees below saturation

Pulsar-WaW agrees with the simulator at 250 and 550 requests/s, and PowerOf2 at 250. At 250, Pulsar-WaW reached 249.6 goodput and 94.0% reuse on the fleet against 249.3 and 94.0% simulated; PowerOf2 reached 200.2 against 200.7.

Fleet goodput at 550 is about 5% below the simulator because fewer turns were offered: 520 requests/s for Pulsar-WaW against 550 simulated. Real network and streaming time lengthen each turn. Pulsar-WaW still met the SLO for 100.0% of what was offered.

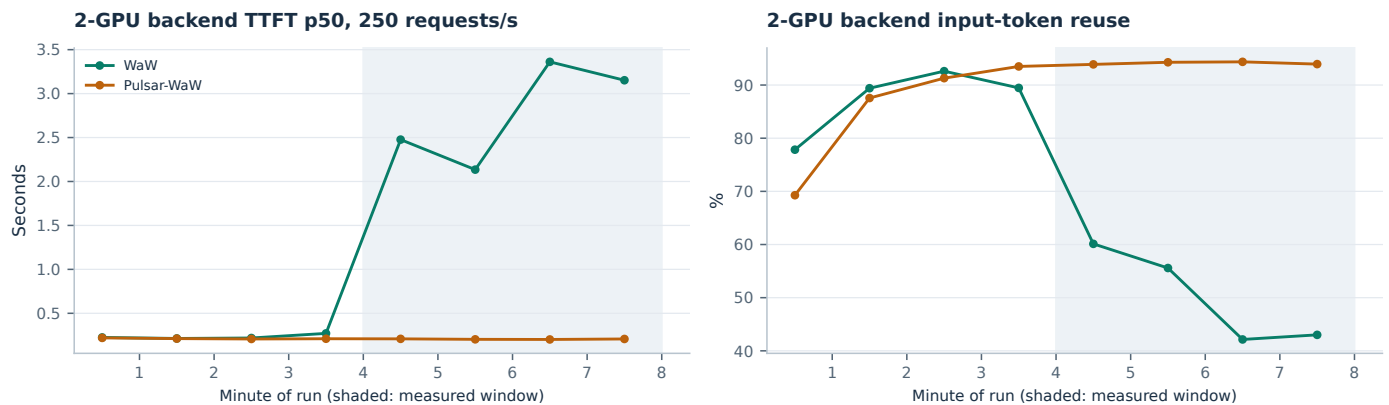


TTFT p50 of each backend against its GPU count. Each point is one backend over the measured window; crosses are simulator seed 1 under WaW.

Small backends are the pressure point. WaW's affinity ring gives each backend an equal share of sessions whatever its size. At 550 requests/s its 2-GPU backend ran at 7.0 s p50 on the fleet and 6.1 s in the simulator, and the 2-4 GPU backends produced all 148 of its fleet timeouts. Pulsar-WaW also slows its 2-GPU backend (2.1 s fleet, 2.5 s simulated) but completed 1,103 requests there in the measured window against 419 for WaW, with no timeouts.

FLEET / SMALL BACKENDS

WaW tips small backends into cache thrash



The 2-GPU backend, one point per minute of the 250 requests/s runs. Arrival minute 4 is the start of the measured window.

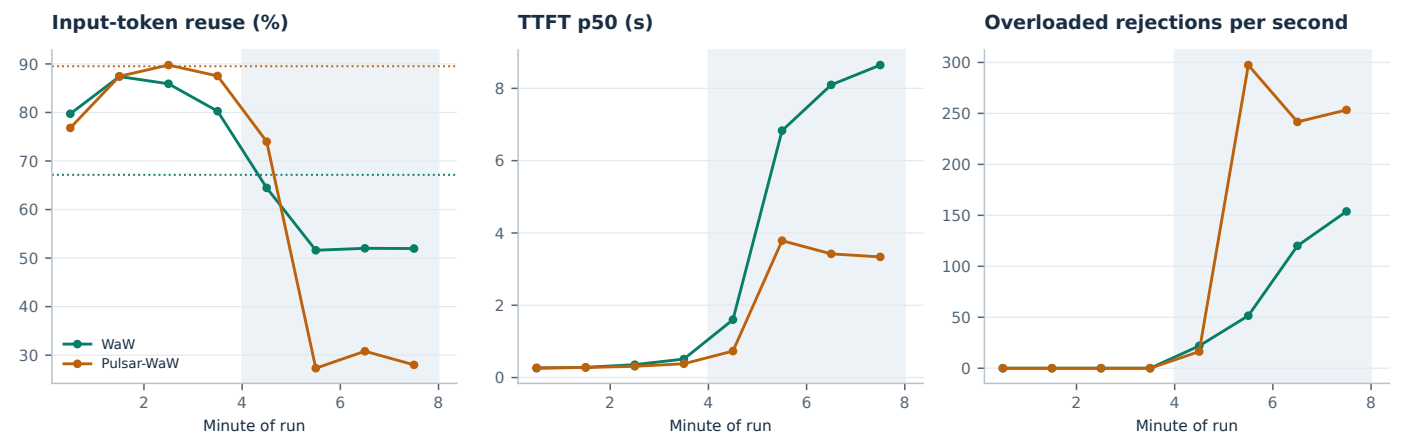
Under WaW the 2-GPU backend served the simulated behavior for three minutes, then switched state in minute 4: TTFT p50 rose to 2.5 s and reuse fell to 60%. It never recovered. Under Pulsar-WaW the same backend stayed at 0.21 s p50 or better with 93%-94% reuse.

This matches the bistable small-backend behavior first seen in the simulator: once evictions start, reuse falls, prefill work grows and the backend stays overloaded. The simulator stayed in the good state at 250 under both seeds; the fleet tipped over. The trigger is not isolated: both policies sent this backend about the same number of requests over the run (3,075 under Pulsar-WaW, 3,049 under WaW), so the difference is in how each policy reacts once the backend slows.

CPU is not the cause: the nodes hosting this backend peaked at 0.88 cores during the WaW measured window, on 2-vCPU nodes.

FLEET / 900 REQUESTS/S

Both affinity policies collapse on the fleet



Per-minute fleet values at a nominal 900 requests/s. Dotted lines in the reuse panel are the simulator seed-1 run-level reuse for each policy.

Pulsar-WaW held 88% reuse through minute 3, then reuse fell fleet-wide to 27% in minute 5 and Stargate began rejecting requests as overloaded in all five regions. Measured goodput was 380 against 861 simulated. WaW collapsed more gradually to 320 goodput, against 572 simulated.

Policy	RPS	Pylon mean cores	Pylon peak	MockDC node samples >= 1.8 cores	MockDC node peak	Stargate peak
WaW	250	0.39	0.73	3%	1.99	0.21
Pulsar-WaW	250	0.53	0.72	2%	1.82	0.21
WaW	550	0.52	0.87	8%	1.98	0.29
Pulsar-WaW	550	0.71	0.86	12%	2.00	0.30
WaW	900	0.50	0.85	5%	1.82	0.71
Pulsar-WaW	900	0.50	0.84	2%	1.83	0.27
PowerOf2	250	0.36	0.73	0%	1.41	0.17

The cause is not confirmed. The fleet had more latency and jitter than the model: real cross-region links, token streaming through reverse tunnels and Pylon CPU. Pylon peaked at 0.84-0.87 cores against a 1-CPU limit in every run from 550 upward, which is consistent with throttling on the busiest backends; throttling counters were not captured for those runs. MockDC nodes are 2-vCPU t3.medium instances.

CONCLUSIONS / EVIDENCE RECORD

Ship candidate: Pulsar-WaW with the tested configuration

What the data supports

- Pulsar-WaW (maximum-TPS weighting, 300 ms affinity wait, 300 ms band widening, max_queued 4, overflow only to free engine slots) is the better of the two shippable policies on both environments.
- Small backends are WaW's weakness on a heterogeneous fleet: they overload first and, once they start evicting, do not recover within a run.
- The simulator is a reliable guide below saturation. It is optimistic near saturation, where the fleet tips into collapse earlier.

Next measurements

- Repeat 700 and 900 requests/s on larger MockDC nodes with Pylon CPU headroom and throttling counters, to separate infrastructure limits from routing behavior.
- Add a second fleet seed and a 700 requests/s pair to locate the fleet collapse point.
- Run round-robin and the 150 requests/s baselines on the fleet; they were skipped for time.
- Repeat the decisive cases on real engines before a production recommendation.

Run record

Item	Detail
Fleet runs	7 accepted runs, seed 1, one run per policy and rate
Excluded	One earlier WaW 900 run: the driver's open-file soft limit of 1,024 caused about 52,000 connect failures. Rerun with the limit raised to 524,288.
Skipped	Round-robin 250, the 700 pair and the 150 baselines, to fit the time budget
Interruptions	A short loss of Kubernetes API access interrupted one backend reset; the run was restarted from a clean reset
Images	Stargate, Pylon and MockDynamo built from one commit of the open routing PR stack; MockDynamo adds the stats-stream switch from this PR

Limitations

MockDynamo models batching, chunked prefill and per-worker KV caching with estimated step costs; it is not calibrated against a real engine. Cache reuse assumes a cached session covers its whole prompt; there is no block-level prefix matching. Load balancers use an unseeded random generator, so simulator runs are not bit-for-bit reproducible. Fleet CPU samples come from the kubelet summary API every 30 s.

Bundle file	Purpose
report.pdf / report.html	Portable report and self-contained browser version
results.json / results.csv	Run-level values, per-backend tables, per-minute series and CPU statistics
graphs/*.svg / *.png	Standalone figures
extract_results.py / build_report.py / render_pdf.py	Extraction, figures and rendering